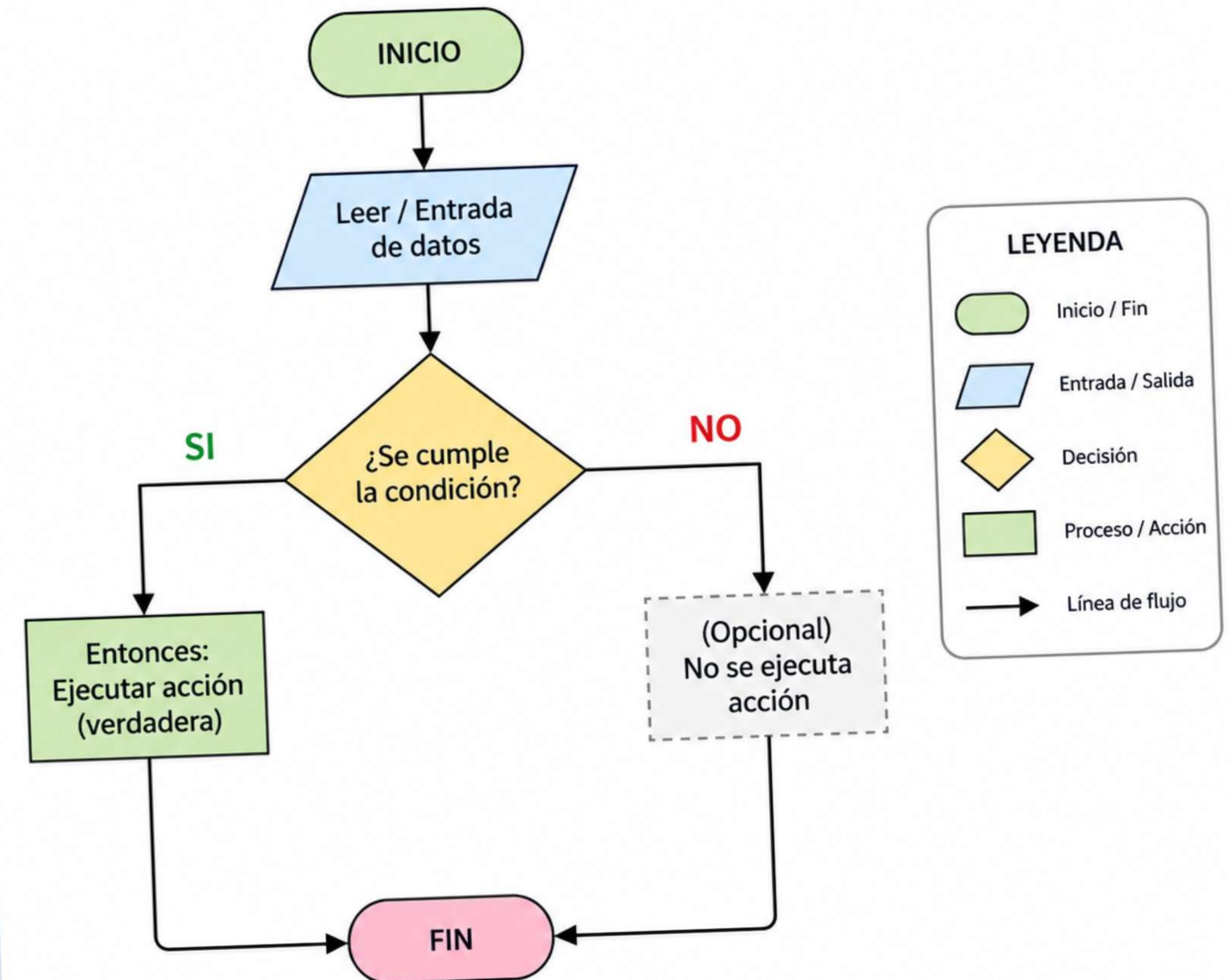


# Pensamiento lógico y programación

Claves para resolver problemas complejos con algoritmos eficientes.

## DIAGRAMA DE FLUJO LÓGICA CONDICIONAL: SI ENTONCES





# Índice

- Pensamiento lógico y programador
- Descomposición de problemas
- Algoritmos y Diagramas de flujo
- Estructuras de control esenciales
- Modularización y Debugging





# Pensamiento Lógico en Programación

El pensamiento lógico no es una capacidad innata, sino un método sistemático que se entrena.

Consiste en **analizar situaciones** para identificar sus componentes, **establecer relaciones** coherentes entre las partes y **generar soluciones** ordenadas y eficientes.



Engranajes: combinando componentes para generar soluciones lógicas.





# Pensamiento común

Cuando nos enfrentamos a un desafío, la mentalidad común tiende a ver el problema como un **bloque sólido**, intentando resolver todo de una vez.

Esto puede generar frustración ante el error, ya que el enfoque principal está en el resultado final, sin apreciar el proceso.

# Pensamiento programador

En contraste, el pensamiento de un programador ve el problema como un **conjunto de piezas** interconectadas, que se resuelven paso a paso (de forma *iterativa*).

Es como resolver un rompecabezas: no encajas todas las piezas al azar, sino que separas bordes, agrupas por colores y construyes secciones pequeñas hasta completar el cuadro.





# ¿Cómo resolver problemas grandes?

Imagina enfrentar un desafío masivo, como **organizar una fiesta**.

Siguiendo nuestra discusión previa sobre la mentalidad del programador:

- ¿Cuál sería tu **primer paso real** para abordar tal complejidad?
- ¿Cómo evitarías sentirte **abrumado** por la magnitud del problema?





# Descomposición: Divide y Vencerás

1

## Definir logística

Identificar fecha, horario y seleccionar el lugar, estableciendo la base del evento.

2

## Gestionar invitados

Crear una lista detallada y proceder con el envío de invitaciones, asegurando la asistencia deseada.

3

## Planificar Catering

Diseñar un menú atractivo y realizar todas las compras necesarias para la comida y bebida.

4

## Coordinar ejecución

Preparar el espacio, organizar las actividades durante la fiesta y gestionar la limpieza posterior, culminando el evento.





# ¿Por qué descomponemos tareas?



## Mayor claridad

Al dividir problemas grandes en partes más pequeñas, reduces la **carga mental**. Esto te permite concentrarte en una sola cosa a la vez, simplificando la comprensión y resolución de cada componente.



## Facilita paralelismo

La descomposición permite que múltiples personas o equipos trabajen en distintas partes del proyecto **simultáneamente**. Esto acelera el desarrollo y la implementación, optimizando el uso de recursos.





# ¿Por qué descomponemos tareas?



## Fomenta reutilización

Las soluciones a problemas pequeños y específicos pueden ser **reutilizadas** en diferentes contextos o en otros proyectos. Esto ahorra tiempo y esfuerzo, promoviendo la eficiencia en la programación y el diseño.



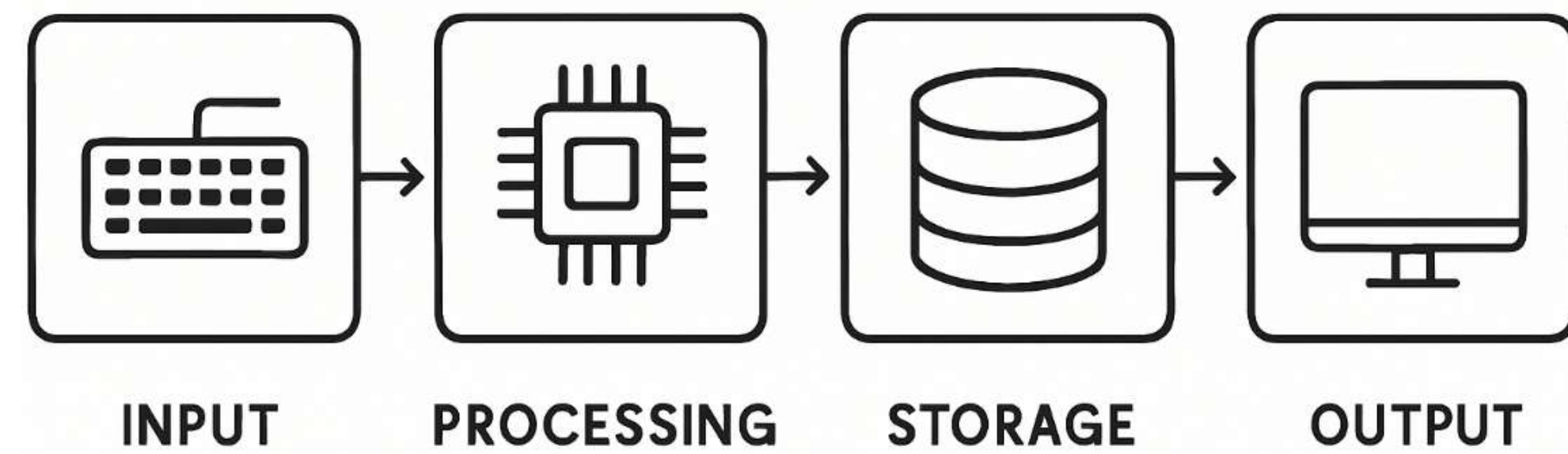
## Mejora prueba

Es considerablemente más fácil encontrar y corregir errores (bugs) en componentes individuales y pequeños. La prueba se vuelve más **precisa** y eficiente, mejorando la calidad general del software.





# ¿Qué es un Algoritmo?



*El flujo de datos: entrada, procesamiento, almacenamiento y salida.*

- Un **algoritmo** es una serie de instrucciones paso a paso para realizar una tarea o resolver un problema.
- Todo algoritmo posee un **Inicio y Fin**, garantizando que siempre debe terminar su ejecución.
- La **Precisión** es clave, ya que cada paso debe estar definido sin ambigüedades, evitando interpretaciones dobles.
- Su **Finitud** implica que debe consistir en un número determinado de pasos, sin bucles infinitos.
- La **Lógica de Decisión** le otorga la capacidad de elegir diferentes caminos según condiciones específicas evaluadas.
- La **Repetición** (o bucles) permite realizar tareas idénticas o similares varias veces, optimizando el código.





# Algoritmo impreciso

Un algoritmo **impreciso** carece de detalles suficientes, generando ambigüedad y resultados impredecibles.

Como vimos en la diapositiva anterior, la *precisión* es clave.

En el caso de cruzar la calle:

1. Llegar a la esquina.
2. Mirar.
3. Cruzar.

*Problema:*

¿Hacia dónde miro? ¿Cruzo si viene un auto?

Esto puede llevar a situaciones peligrosas o errores, ya que las instrucciones no son lo suficientemente claras para todas las condiciones.

# Algoritmo preciso

Un algoritmo **preciso** define cada paso de forma inequívoca, garantizando la *finitud* y resultados consistentes.

La solución lógica para cruzar la calle es:

1. INICIO
2. Llegar a la esquina y detenerse.
3. Mirar a la izquierda, luego a la derecha.
4. **SI** no vienen vehículos: Cruzar.
5. **SINO**: Esperar y repetir el paso 3.
6. Llegar a la vereda opuesta.
7. FIN

Este ejemplo ilustra cómo los detalles eliminan la ambigüedad, un principio fundamental en la programación.





# Diagramas de flujo: Símbolos

Los diagramas de flujo son **representaciones gráficas** de un algoritmo.

- **Óvalo:** Indica el *Inicio* o *Fin* del proceso.
- **Rectángulo:** Representa una *acción* o instrucción (ej. 'Caminar').
- **Rombo:** Señala una *decisión* (pregunta SI/NO).
- **Paralelogramo:** Para *Entrada* de datos o *Salida* de información.
- **Flechas:** Muestran la *dirección* del flujo lógico.





# Diagrama: cruzar la calle

Para entender cómo funcionan los diagramas de flujo, consideremos el proceso de **cruzar la calle**.

Siguiendo la simbología que vimos, el flujo inicia, se llega a la esquina, y un rombo evalúa "¿Viene algún auto?". Si la respuesta es **SÍ**, se espera; si es **NO**, se procede a cruzar, y el flujo termina.

Estos diagramas son cruciales para detectar errores lógicos *antes de programar*.



Señal de cruce peatonal: un ejemplo para diagramas de flujo.

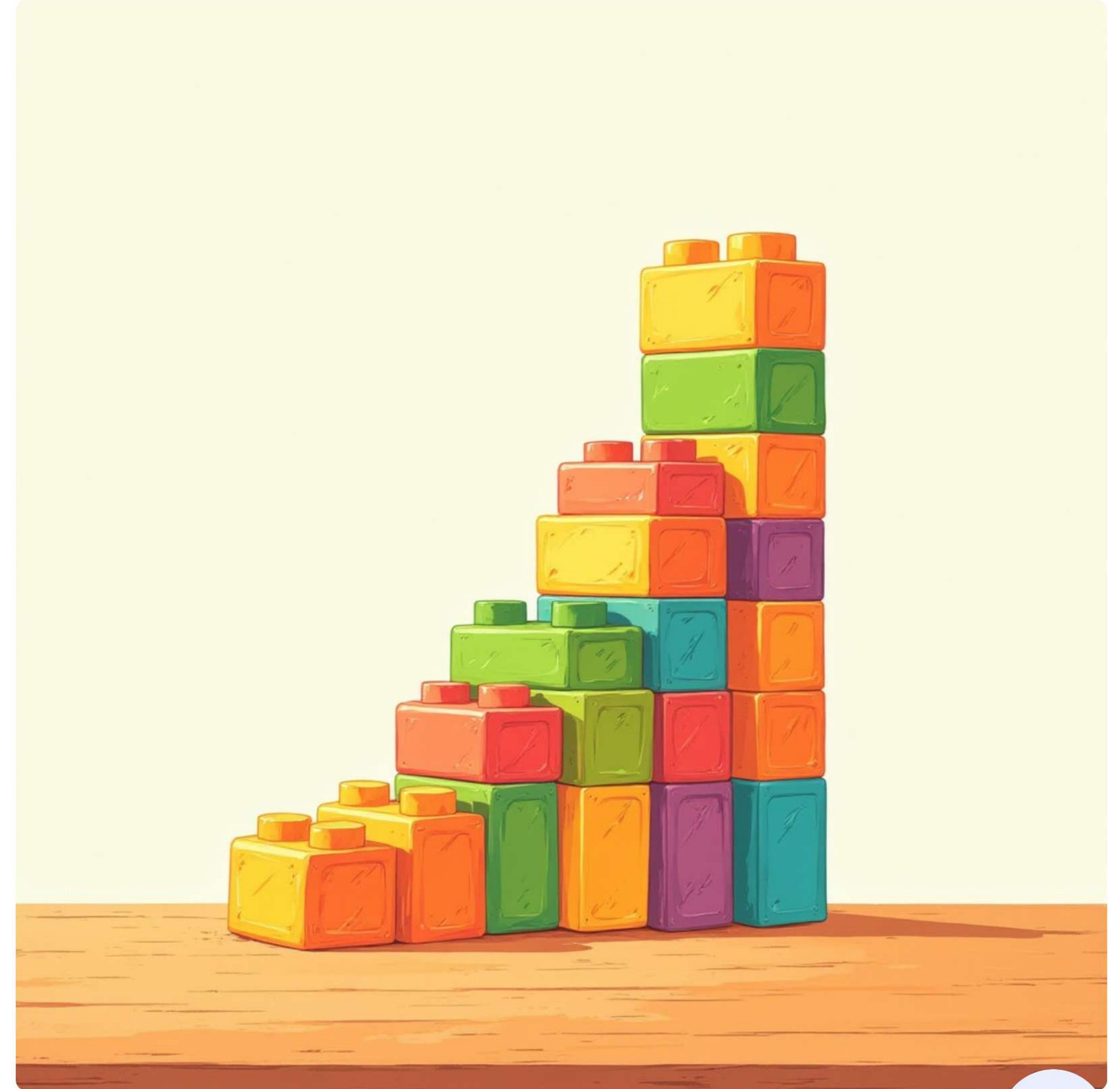




# Estructuras de control: secuencia

La **estructura secuencial** es la más simple en programación, donde las instrucciones se ejecutan una tras otra, en el orden en que aparecen. Es como una lista de tareas donde cada paso debe completarse antes de pasar al siguiente, tal como los pasos de un algoritmo que vimos anteriormente.

Un ejemplo claro es una rutina de entrenamiento: primero el **Calentamiento**, luego la **Rutina** de ejercicios, y finalmente el **Enfriamiento**. El orden es crucial para el resultado deseado.





# Estructura Selectiva (IF/SI)



## Condición inicial

Si la **edad** es menor de 18 ( $\text{edad} < 18$ ), el resultado es "Acceso denegado". Esta es la primera comprobación del sistema.



## Condición intermedia

En caso contrario (SINO), si la **edad** es menor o igual a 21 ( $\text{edad} \leq 21$ ), el resultado es "Pasa con pulsera de menor". Se evalúa solo si la primera condición es falsa.





# Estructura Selectiva (IF/SI)



## Condición final

Si ninguna de las condiciones anteriores se cumple (SINO), el resultado es "Pasa libre". Esta es la acción por defecto cuando las comprobaciones previas son falsas.



## Árbol de decisiones

A diferencia de la *estructura secuencial*, esta estructura bifurca el flujo del programa, creando un **árbol de decisiones**. Solo un camino se ejecuta por cada entrada, optimizando el procesamiento.





# Estructuras Repetitivas (WHILE)

1

## Introducción a los bucles

Los **bucles o ciclos** son estructuras que permiten repetir una o varias acciones un número determinado de veces o *mientras* una condición específica se mantenga **verdadera**.

2

## Establecer la Inicialización

Antes de que el bucle comience, es crucial *inicializar* una variable de control. Esta variable se utiliza para determinar cuándo debe terminar el ciclo.

3

## Evaluación de la Condición

En cada *iteración* (repetición) del bucle, se evalúa una condición lógica. El bucle `WHILE` continuará ejecutándose **MIENTRAS** la condición sea verdadera.





# Estructuras Repetitivas (WHILE)

4

## Ejecución de la acción

Si la condición evaluada es verdadera, el bloque de código dentro del bucle se ejecuta.

Esta es la *acción* que queremos repetir.

En nuestro ejemplo de sentadillas, la acción es 'Hacer 10 sentadillas'.

5

## Actualización de la variable

Después de ejecutar la acción, la variable de control debe ser *actualizada*. Esto es vital para asegurar que la condición eventualmente se vuelva falsa y el bucle termine, evitando un *bucle infinito*.





# Modularización y funciones

La modularización consiste en dividir un programa en partes independientes llamadas **módulos o funciones**.

Cada módulo realiza una tarea específica, simplificando la complejidad del programa global.

## Ventajas

- **Reutilización:** Escribes el código una vez y lo usas muchas veces, lo que ahorra tiempo y esfuerzo.
- **Mantenimiento:** Si algo falla, solo arreglas ese módulo específico sin afectar otras partes del programa.
- **Colaboración:** Diferentes programadores pueden trabajar en distintos módulos a la vez, agilizando el desarrollo.

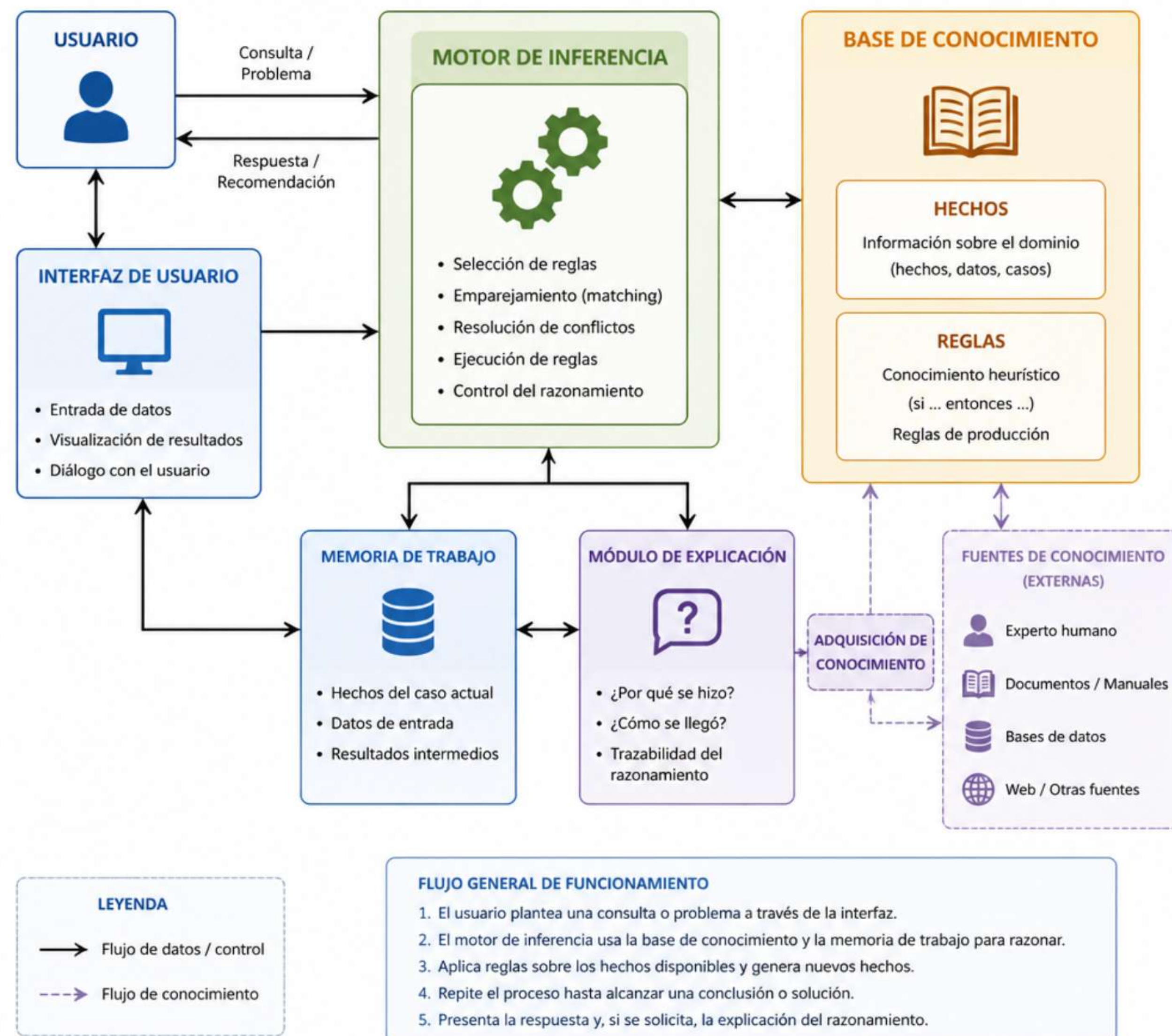
## Analogía práctica

Imagina un equipo de fútbol: el portero, los defensas y los delanteros tienen funciones específicas. Su trabajo en conjunto, pero dividido, hace al equipo funcional y exitoso.





## ARQUITECTURA DE UN SISTEMA EXPERTO



# Ejemplo de rutina modular

En lugar de un código gigante y desordenado, creamos un **Programa Principal** que orchestra la ejecución de subprocesos específicos.

Cada uno de estos subprocesos actúa como una 'caja negra' independiente que sabe exactamente qué tarea realizar sin interferir con el resto del programa, demostrando la eficiencia de la modularización.





# Debugging: el arte de corregir



## Definición y Propósito

El **Debugging** (depuración) es el proceso fundamental de encontrar y eliminar *errores* (o 'bugs') en el código. En programación, estos errores no son fallos, sino **pistas cruciales** para mejorar la lógica y funcionalidad del programa.

Es una habilidad esencial para asegurar que los módulos de nuestro programa principal funcionen correctamente y coordinados.



## Error de lógica

Se produce cuando el programa se ejecuta sin interrupciones, pero el resultado obtenido es *incorrecto*.

Esto puede deberse a una operación matemática errónea (ej. sumar cuando se debía restar) o una evaluación condicional mal planteada.





# Debugging: el arte de corregir



## Error de Secuencia

Este tipo de error surge cuando los pasos de un algoritmo o las instrucciones de un programa se realizan en un **orden equivocado**, lo que lleva a un comportamiento inesperado o a un resultado no deseado.



## Error de Condición

Relacionado con *bucles* (loops) o *estructuras condicionales* mal formuladas. Un ejemplo clásico es un **bucle infinito**, donde la condición de salida nunca se cumple, causando que el programa se quede atascado en una ejecución repetitiva sin fin.





# El camino del pensador

## Descomposición y Algoritmos

Comienza por **dividir** problemas complejos en partes más manejables. Luego, define **algoritmos** (pasos precisos y lógicos) para resolver cada segmento, de forma similar a cómo detectamos errores de secuencia en el ejercicio de debugging.

## Estructuras y Control

Aprende a dominar el *flujo de control* de tus programas. Esto se logra con estructuras de decisión (**Si**) que permiten elegir caminos, y **ciclos (Mientras)** que repiten acciones, gestionando la lógica del proceso.

## Modularización

Organiza tu código en **módulos** (piezas independientes y reutilizables) o funciones. Esto no solo facilita la lectura y mantenimiento, sino que también fomenta la eficiencia y la colaboración en proyectos grandes.



# ¿Tu próximo problema?

Después de explorar herramientas algorítmicas, ¿qué proceso de tu vida diaria (estudio, deporte, hobby) te gustaría modelar como un **algoritmo** o **diagrama de flujo** para optimizarlo o comprenderlo mejor?

